# Editor de imágenes con MFC y OpenCV 5.1

**Compilación de OpenCV 5.x con opencv_contrib en Windows y su uso en una aplicación de escritorio MFC**

| Dato | Valor |
|---|---|
| Alumno | Josué Daniel Ayala Mejía |
| Matrícula | 1240116 |
| Materia | Programación del sistema |
| Profesor | Ignacio Cruz Domínguez |
| Institución | Universidad de Montemorelos |
| Fecha de entrega | 14/10/2026 |

## 1. Introducción y objetivo

Este documento describe cómo se compiló la librería OpenCV 5.1 junto con opencv_contrib a partir de su código fuente, usando CMake y Visual Studio 2026 en Windows (x64), y cómo se utilizó en un editor de imágenes con interfaz gráfica MFC. En la aplicación, la interfaz controla los parámetros (botones y sliders) y OpenCV procesa la imagen.

Funciones del editor: abrir y guardar imágenes, brillo, contraste, desenfoque gaussiano, escala de grises, detección de bordes (Canny), volteo y restauración de la imagen original.

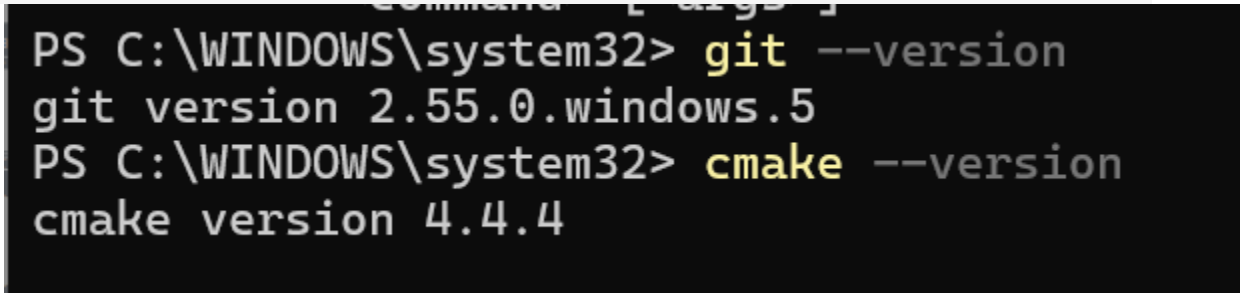
## 2. Requisitos y herramientas

| Requisito | Detalle |
|---|---|
| Sistema | Windows 10 u 11, 64 bits |
| Visual Studio | Community 2026, con la carga de trabajo **Desarrollo para el escritorio con C++** |
| MFC | Componente **MFC de C++ para x64/x86 (MSVC más reciente)** |
| CMake | Versión 4.2 o superior (necesaria para el generador `Visual Studio 18 2026`) |
| Git | Para descargar el código fuente |
| Espacio y tiempo | Unos 15 GB libres; la compilación tarda entre 30 y 90 minutos |

Si CMake o Git no están instalados, se instalan con winget y se reabre PowerShell:

```
winget install Kitware.CMake
winget install Git.Git
```
Verificación de las herramientas:

```
git --version
cmake --version
& "${env:ProgramFiles(x86)}\Microsoft Visual Studio\Installer\vswhere.exe" -latest
-property displayName
```




## 3. Compilación de OpenCV 5.x

### 3.1 Carpetas y script

```
New-Item -ItemType Directory -Force -Path C:\cv5, C:\img
```
El archivo `C:\cv5\build-opencv5.ps1` contiene los comandos que descargan, configuran, compilan e instalan OpenCV. Es una versión resumida del script de clase, con el generador de Visual Studio 2026 y sin ejemplos ni aplicaciones para acelerar la compilación:

```
$ErrorActionPreference = "Continue"

$Root       = "C:/cv5"
$OpenCVDir  = "$Root/opencv"
$ContribDir = "$Root/opencv_contrib"
$BuildDir   = "$Root/build"
$InstallDir = "$Root/install"
$Generator  = "Visual Studio 18 2026"

Write-Host "1. Descargando OpenCV..." -ForegroundColor Cyan
if (-not (Test-Path "$OpenCVDir/.git")) {
    git clone --branch 5.x --depth 1 https://github.com/opencv/opencv.git $OpenCVDir
}

Write-Host "2. Descargando opencv_contrib..." -ForegroundColor Cyan
if (-not (Test-Path "$ContribDir/.git")) {
    git clone --branch 5.x --depth 1 https://github.com/opencv/opencv_contrib.git
$ContribDir
}

Write-Host "3. Preparando carpeta build..." -ForegroundColor Cyan
if (Test-Path $BuildDir) { Remove-Item $BuildDir -Recurse -Force }
New-Item -ItemType Directory -Force -Path $BuildDir | Out-Null
Set-Location $BuildDir

Write-Host "4. Configurando CMake..." -ForegroundColor Cyan
```

```
$CMakeArgs = @(
    "-G", $Generator, "-A", "x64",
    "-DCMAKE_CXX_STANDARD=17",
    "-DCMAKE_CXX_STANDARD_REQUIRED=ON",
    "-DBUILD_WITH_DEBUG_INFO=ON",
    "-DOPENCV_EXTRA_MODULES_PATH=$ContribDir/modules",
    "-DBUILD_TESTS=OFF",
    "-DBUILD_PERF_TESTS=OFF",
    "-DBUILD_EXAMPLES=OFF",
    "-DBUILD_DOCS=OFF",
    "-DBUILD_opencv_apps=OFF",
    "-DBUILD_opencv_world=ON",
    "-DBUILD_opencv_python3=OFF",
    "-DBUILD_opencv_java=OFF",
    "-DWITH_VTK=OFF",
    "-DWITH_OPENCL=OFF",
    "-DWITH_TBB=OFF",
    "-DWITH_OPENMP=OFF",
    "-DWITH_FFMPEG=ON",
    "-DCMAKE_INSTALL_PREFIX=$InstallDir",
    $OpenCVDir
)
cmake @CMakeArgs
if ($LASTEXITCODE -ne 0) { throw "La configuracion de CMake fallo." }

$Jobs = [Environment]::ProcessorCount

Write-Host "5. Compilando Release (esto tarda)..." -ForegroundColor Cyan
cmake --build . --config Release --parallel $Jobs
if ($LASTEXITCODE -ne 0) { throw "La compilacion fallo." }

Write-Host "6. Instalando..." -ForegroundColor Cyan
cmake --build . --config Release --target INSTALL --parallel $Jobs
if ($LASTEXITCODE -ne 0) { throw "La instalacion fallo." }

Write-Host "7. Verificando..." -ForegroundColor Cyan
Get-ChildItem $InstallDir -Recurse -Filter "opencv_world*.lib" | Select-Object
FullName
Get-ChildItem $InstallDir -Recurse -Filter "opencv_world*.dll" | Select-Object
FullName
Write-Host "TERMINADO" -ForegroundColor Green
```

### 3.2 Qué hace cada paso

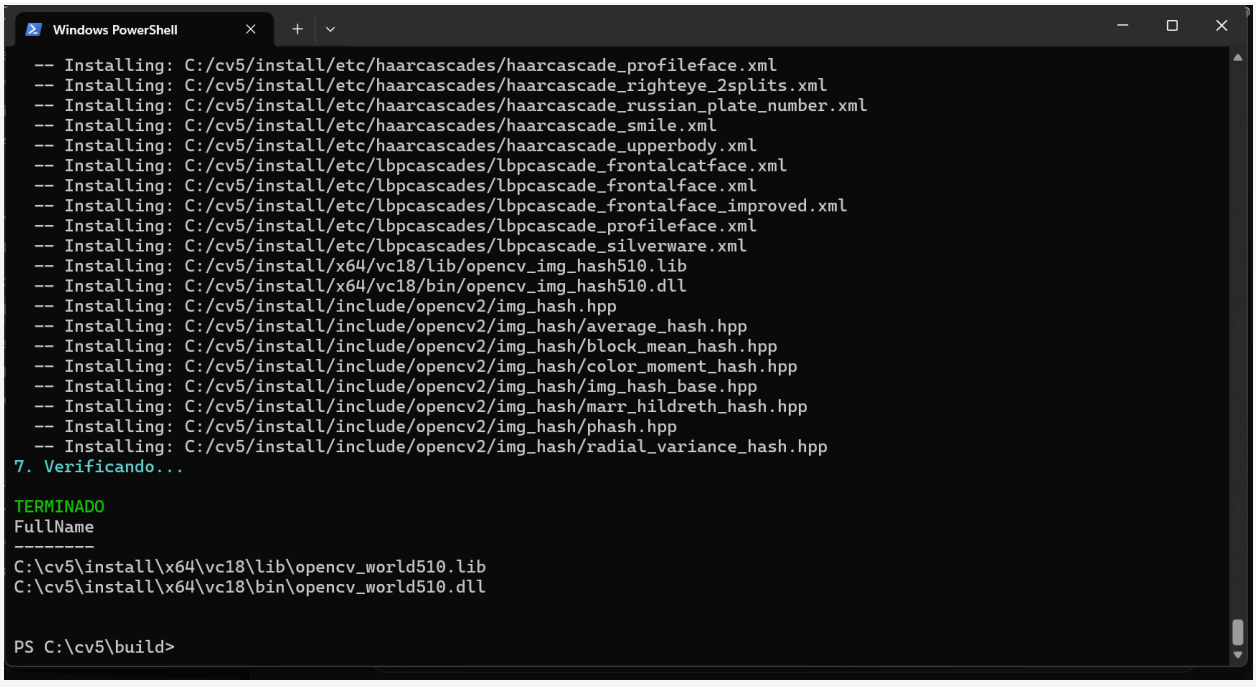
| Paso | Acción | Resultado |
|---|---|---|
| 1 y 2 | `git clone` de `opencv` y `opencv_contrib` (rama 5.x) | Código fuente en `C:\cv5\opencv` y `C:\cv5\opencv_contrib` |
| 3 | Limpia y crea la carpeta `build` | Compilación separada del código fuente |

| Paso | Acción | Resultado |
|---|---|---|
| 4 | CMake genera la solución de Visual Studio | Proyecto para x64 y C++17, con módulos contrib y un solo `opencv_world` |
| 5 | Compilación en Release | Biblioteca y DLL generadas |
| 6 | Destino `INSTALL` | Encabezados, `.lib` y `.dll` copiados a `C:\cv5\install` |
| 7 | Búsqueda de la `.lib` y la `.dll` | Rutas finales y mensaje TERMINADO |

### 3.3 Ejecución

```
Set-ExecutionPolicy -Scope Process Bypass
cd C:\cv5
.\build-opencv5.ps1
```

Al terminar, el script imprime las rutas de la biblioteca y la DLL y la palabra TERMINADO.

> *[*
>
> 
> 

### 3.4 Resultado de la instalación

| Qué | Ruta |
|---|---|
| Encabezados | `C:\cv5\install\include\opencv2` |
| Biblioteca | `C:\cv5\install\x64\vc18\lib\opencv_world510.lib` |

| Qué | Ruta |
|---|---|
| DLL | `C:\cv5\install\x64\vc18\bin\opencv_world510.dll` |

## 4. Proyecto MFC

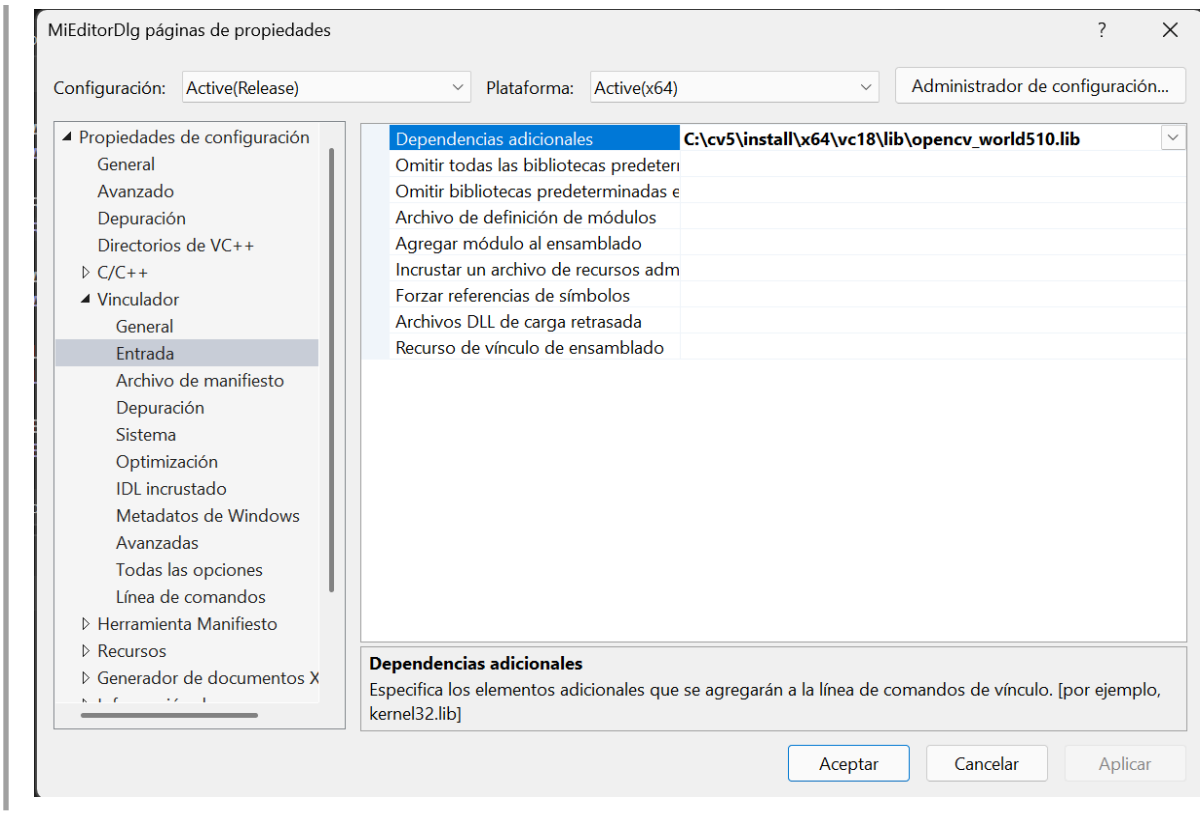
### 4.1 Creación

En Visual Studio: **Crear un proyecto nuevo** → **Aplicación MFC** → nombre `MiEditorDlg`. En el asistente, **Tipo de aplicación: Basado en cuadro de diálogo** y **MFC en una DLL compartida**. Se generan la clase de la aplicación `CMiEditorDlgApp` y la clase del diálogo `CMiEditorDlgDlg` (`MiEditorDlgDlg.h` y `.cpp`), donde va todo el código del editor.

### 4.2 Configuración Release | x64

La biblioteca se compiló solo en Release y x64, así que el proyecto se configura igual. En la barra superior se elige **Release** y **x64**, y en **Propiedades del proyecto**:

| Ubicación | Valor |
|---|---|
| C/C++ → General → Directorios de inclusión adicionales | `C:\cv5\install\include` |
| C/C++ → Lenguaje → Estándar del lenguaje C++ | ISO C++17 |
| Vinculador → General → Directorios de bibliotecas adicionales | `C:\cv5\install\x64\vc18\lib` |
| Vinculador → Entrada → Dependencias adicionales | `opencv_world510.lib` |




### 4.3 Encabezado precompilado (pch.h)

OpenCV se incluye antes de `framework.h`, con `NOMINMAX` para evitar el conflicto de MFC con `min` y `max`:

```
#ifndef PCH_H
#define PCH_H

#define NOMINMAX
#include <opencv2/opencv.hpp>
#include <vector>
#include <algorithm>

#include "framework.h"

#endif //PCH_H
```

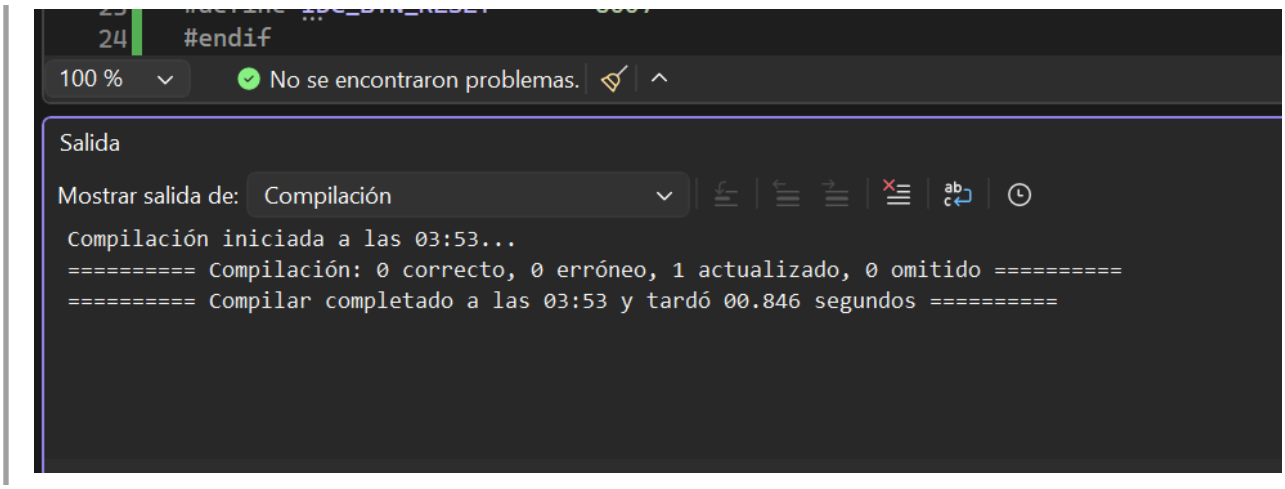
### 4.4 DLL de OpenCV

La aplicación necesita `opencv_world510.dll` junto a su `.exe`:

```
Copy-Item "C:\cv5\install\x64\vc18\bin\opencv_world510.dll"
"C:\Users\Danny\source\repos\MiEditorDlg\x64\Release\"
```

### 4.5 Verificación de la librería

Al compilar con **Ctrl+Shift+B** el resultado fue `1 correcto, 0 erróneo`. Al ejecutar con **F5**, la ventana Salida muestra que la aplicación cargó la DLL de OpenCV.




## 5. Interfaz gráfica creada por código

El diálogo que genera Visual Studio solo trae el texto "TODO" y los botones Aceptar y Cancelar. En lugar de arrastrar controles en el editor de recursos, la interfaz se construye en tiempo de ejecución dentro de `OnInitDialog()`, con el método `Create()` de cada clase de control MFC.

En `OnInitDialog()` se oculta el botón Aceptar y se elimina el texto "TODO", se redimensiona la ventana a 900 × 500, se crean los controles, se configuran los rangos de los sliders, se reutiliza Cancelar como botón **Salir** y se muestra la versión de OpenCV (`CV_VERSION`) en la barra de título.

| Control | Clase MFC | ID | Posición y rango |
|---|---|---|---|
| Imagen | `CStatic` | `IDC_PIC` | Izquierda, 640 × 480 px |
| Abrir, Guardar, Grises, Bordes, Voltear, Restaurar | `CButton` | `IDC_BTN_OPEN`, `IDC_BTN_SAVE`, `IDC_BTN_GRAY`, `IDC_BTN_CANNY`, `IDC_BTN_FLIP`, `IDC_BTN_RESET` | Columna derecha, uno cada 38 px |
| Brillo | `CSliderCtrl` | `IDC_SLIDER_BRIGHT` | −100 a 100 |
| Contraste | `CSliderCtrl` | `IDC_SLIDER_CONTRAST` | 1 a 30 (se divide entre 10: 0.1 a 3.0) |
| Desenfoque | `CSliderCtrl` | `IDC_SLIDER_BLUR` | 0 a 20 |
| Salir | `CButton` (el Cancelar original) | `IDCANCEL` | Parte inferior derecha |

Los IDs se definen en `MiEditorDlgDlg.h` con `#ifndef`, así no chocan con los de `Resource.h`.

## 6. Código de la aplicación

Se reemplaza el contenido completo de los dos archivos del diálogo.

### 6.1 MiEditorDlgDlg.h

```
#pragma once

// IDs de los controles (se definen aquí; si ya existen en Resource.h, se respetan)
#ifndef IDC_PIC
#define IDC_PIC             5001
#endif
#ifndef IDC_BTN_OPEN
#define IDC_BTN_OPEN        5002
#endif
#ifndef IDC_BTN_SAVE
#define IDC_BTN_SAVE        5003
#endif
#ifndef IDC_BTN_GRAY
#define IDC_BTN_GRAY        5004
#endif
#ifndef IDC_BTN_CANNY
#define IDC_BTN_CANNY       5005
#endif
#ifndef IDC_BTN_FLIP
#define IDC_BTN_FLIP        5006
#endif
#ifndef IDC_BTN_RESET
#define IDC_BTN_RESET       5007
#endif
#ifndef IDC_SLIDER_BRIGHT
#define IDC_SLIDER_BRIGHT   5008
#endif
#ifndef IDC_SLIDER_CONTRAST
#define IDC_SLIDER_CONTRAST 5009
#endif
#ifndef IDC_SLIDER_BLUR
#define IDC_SLIDER_BLUR     5010
#endif

class CMiEditorDlgDlg : public CDialogEx
{
public:
	CMiEditorDlgDlg(CWnd* pParent = nullptr);

#ifdef AFX_DESIGN_TIME
	enum { IDD = IDD_MIEDITORDLG_DIALOG };
#endif

protected:
	virtual void DoDataExchange(CDataExchange* pDX);
	virtual BOOL OnInitDialog();
	virtual void OnOK() {}   // evita que Enter cierre la aplicación

	HICON m_hIcon;

	// Controles creados por código
	CStatic     m_pic;
	CButton     m_btnOpen, m_btnSave, m_btnGray, m_btnCanny, m_btnFlip,
```

```
m_btnReset;
	CSliderCtrl m_sliderBright, m_sliderContrast, m_sliderBlur;
	CStatic     m_lblBright, m_lblContrast, m_lblBlur;

	// Datos de OpenCV
	cv::Mat m_original;   // imagen cargada, nunca se modifica
	cv::Mat m_current;    // imagen con los cambios aplicados
	bool m_gray  = false;
	bool m_edges = false;
	bool m_flip  = false;

	void ApplyPipeline();
	void ShowImage(const cv::Mat& src);

	// Mensajes
	afx_msg void OnSysCommand(UINT nID, LPARAM lParam);
	afx_msg void OnPaint();
	afx_msg HCURSOR OnQueryDragIcon();
	afx_msg void OnHScroll(UINT nSBCode, UINT nPos, CScrollBar* pScrollBar);
	afx_msg void OnBnClickedBtnOpen();
	afx_msg void OnBnClickedBtnSave();
	afx_msg void OnBnClickedBtnGray();
	afx_msg void OnBnClickedBtnCanny();
	afx_msg void OnBnClickedBtnFlip();
	afx_msg void OnBnClickedBtnReset();
	DECLARE_MESSAGE_MAP()
};
```

### 6.2 MiEditorDlgDlg.cpp

```
#include "pch.h"
#include "framework.h"
#include "MiEditorDlg.h"
#include "MiEditorDlgDlg.h"
#include "afxdialogex.h"

#ifdef _DEBUG
#define new DEBUG_NEW
#endif

// ---------------- Cuadro "Acerca de" ----------------
class CAboutDlg : public CDialogEx
{
public:
	CAboutDlg();
#ifdef AFX_DESIGN_TIME
	enum { IDD = IDD_ABOUTBOX };
#endif
protected:
	virtual void DoDataExchange(CDataExchange* pDX);
	DECLARE_MESSAGE_MAP()
};

CAboutDlg::CAboutDlg() : CDialogEx(IDD_ABOUTBOX) {}
void CAboutDlg::DoDataExchange(CDataExchange* pDX) { CDialogEx::DoDataExchange(pDX);
```

```
}
BEGIN_MESSAGE_MAP(CAboutDlg, CDialogEx)
END_MESSAGE_MAP()

// ---------------- Diálogo principal ----------------
CMiEditorDlgDlg::CMiEditorDlgDlg(CWnd* pParent /*=nullptr*/)
	: CDialogEx(IDD_MIEDITORDLG_DIALOG, pParent)
{
	m_hIcon = AfxGetApp()->LoadIcon(IDR_MAINFRAME);
}

void CMiEditorDlgDlg::DoDataExchange(CDataExchange* pDX)
{
	CDialogEx::DoDataExchange(pDX);
	// Los controles se crean por código en OnInitDialog
}

BEGIN_MESSAGE_MAP(CMiEditorDlgDlg, CDialogEx)
	ON_WM_SYSCOMMAND()
	ON_WM_PAINT()
	ON_WM_QUERYDRAGICON()
	ON_WM_HSCROLL()
	ON_BN_CLICKED(IDC_BTN_OPEN,  &CMiEditorDlgDlg::OnBnClickedBtnOpen)
	ON_BN_CLICKED(IDC_BTN_SAVE,  &CMiEditorDlgDlg::OnBnClickedBtnSave)
	ON_BN_CLICKED(IDC_BTN_GRAY,  &CMiEditorDlgDlg::OnBnClickedBtnGray)
	ON_BN_CLICKED(IDC_BTN_CANNY, &CMiEditorDlgDlg::OnBnClickedBtnCanny)
	ON_BN_CLICKED(IDC_BTN_FLIP,  &CMiEditorDlgDlg::OnBnClickedBtnFlip)
	ON_BN_CLICKED(IDC_BTN_RESET, &CMiEditorDlgDlg::OnBnClickedBtnReset)
END_MESSAGE_MAP()

BOOL CMiEditorDlgDlg::OnInitDialog()
{
	CDialogEx::OnInitDialog();

	// Elemento "Acerca de..." en el menú del sistema
	ASSERT((IDM_ABOUTBOX & 0xFFF0) == IDM_ABOUTBOX);
	ASSERT(IDM_ABOUTBOX < 0xF000);
	CMenu* pSysMenu = GetSystemMenu(FALSE);
	if (pSysMenu != nullptr)
	{
		CString strAboutMenu;
		BOOL bNameValid = strAboutMenu.LoadString(IDS_ABOUTBOX);
		ASSERT(bNameValid);
		if (!strAboutMenu.IsEmpty())
		{
			pSysMenu->AppendMenu(MF_SEPARATOR);
			pSysMenu->AppendMenu(MF_STRING, IDM_ABOUTBOX, strAboutMenu);
		}
	}

	SetIcon(m_hIcon, TRUE);
	SetIcon(m_hIcon, FALSE);

	// ---------- Limpiar el diálogo vacío (texto TODO, botón Aceptar) ----------
	for (CWnd* p = GetWindow(GW_CHILD); p != nullptr; )
```

```
	{
		CWnd* next = p->GetNextWindow();
		int id = p->GetDlgCtrlID();
		if (id == IDOK)		p->ShowWindow(SW_HIDE);
		else if (id == -1)	p->DestroyWindow();   // texto "TODO"
		p = next;
	}

	// ---------- Tamaño de la ventana ----------
	CRect rcWin(0, 0, 900, 500);
	AdjustWindowRectEx(&rcWin, GetStyle(), FALSE, GetExStyle());
	SetWindowPos(nullptr, 0, 0, rcWin.Width(), rcWin.Height(),
		     SWP_NOMOVE | SWP_NOZORDER);
	CenterWindow();

	CFont* pFont = GetFont();
	const DWORD wsBtn = WS_CHILD | WS_VISIBLE | WS_TABSTOP | BS_PUSHBUTTON;
	const DWORD wsSld = WS_CHILD | WS_VISIBLE | WS_TABSTOP | TBS_HORZ |
TBS_AUTOTICKS;
	const DWORD wsLbl = WS_CHILD | WS_VISIBLE | SS_LEFT;

	// ---------- Cuadro de imagen (640 x 480) ----------
	m_pic.Create(_T(""), WS_CHILD | WS_VISIBLE | WS_BORDER | SS_BITMAP |
SS_CENTERIMAGE,
		     CRect(10, 10, 650, 490), this, IDC_PIC);

	// ---------- Botones ----------
	struct BtnInfo { CButton* b; UINT id; const TCHAR* text; };
	BtnInfo btns[] = {
		{ &m_btnOpen,  IDC_BTN_OPEN,  _T("Abrir")     },
		{ &m_btnSave,  IDC_BTN_SAVE,  _T("Guardar")   },
		{ &m_btnGray,  IDC_BTN_GRAY,  _T("Grises")    },
		{ &m_btnCanny, IDC_BTN_CANNY, _T("Bordes")    },
		{ &m_btnFlip,  IDC_BTN_FLIP,  _T("Voltear")   },
		{ &m_btnReset, IDC_BTN_RESET, _T("Restaurar") },
	};
	for (int i = 0; i < 6; i++)
	{
		int y = 10 + i * 38;
		btns[i].b->Create(btns[i].text, wsBtn, CRect(670, y, 890, y + 32), this,
btns[i].id);
		btns[i].b->SetFont(pFont);
	}

	// ---------- Sliders con sus etiquetas ----------
	m_lblBright.Create(_T("Brillo"), wsLbl, CRect(670, 250, 890, 268), this);
	m_sliderBright.Create(wsSld, CRect(670, 268, 890, 298), this,
IDC_SLIDER_BRIGHT);

	m_lblContrast.Create(_T("Contraste"), wsLbl, CRect(670, 310, 890, 328), this);
	m_sliderContrast.Create(wsSld, CRect(670, 328, 890, 358), this,
IDC_SLIDER_CONTRAST);

	m_lblBlur.Create(_T("Desenfoque"), wsLbl, CRect(670, 370, 890, 388), this);
	m_sliderBlur.Create(wsSld, CRect(670, 388, 890, 418), this, IDC_SLIDER_BLUR);
```

```
	m_lblBright.SetFont(pFont);
	m_lblContrast.SetFont(pFont);
	m_lblBlur.SetFont(pFont);

	m_sliderBright.SetRange(-100, 100);  m_sliderBright.SetPos(0);
	m_sliderContrast.SetRange(1, 30);    m_sliderContrast.SetPos(10);
	m_sliderBlur.SetRange(0, 20);        m_sliderBlur.SetPos(0);

	// ---------- Botón Cancelar convertido en "Salir" ----------
	CWnd* pCancel = GetDlgItem(IDCANCEL);
	if (pCancel)
	{
		pCancel->SetWindowText(_T("Salir"));
		pCancel->MoveWindow(670, 450, 220, 32);
		pCancel->BringWindowToTop();
	}

	// Versión de OpenCV en la barra de título (evidencia de que la librería
funciona)
	SetWindowText(CString(_T("Editor de imagenes - OpenCV ")) +
CString(CV_VERSION));

	return TRUE;
}

void CMiEditorDlgDlg::OnSysCommand(UINT nID, LPARAM lParam)
{
	if ((nID & 0xFFF0) == IDM_ABOUTBOX)
	{
		CAboutDlg dlgAbout;
		dlgAbout.DoModal();
	}
	else
	{
		CDialogEx::OnSysCommand(nID, lParam);
	}
}

void CMiEditorDlgDlg::OnPaint()
{
	if (IsIconic())
	{
		CPaintDC dc(this);
		SendMessage(WM_ICONERASEBKGND,
reinterpret_cast<WPARAM>(dc.GetSafeHdc()), 0);

		int cxIcon = GetSystemMetrics(SM_CXICON);
		int cyIcon = GetSystemMetrics(SM_CYICON);
		CRect rect;
		GetClientRect(&rect);
		int x = (rect.Width() - cxIcon + 1) / 2;
		int y = (rect.Height() - cyIcon + 1) / 2;
		dc.DrawIcon(x, y, m_hIcon);
	}
```

```
	else
	{
		CDialogEx::OnPaint();
	}
}

HCURSOR CMiEditorDlgDlg::OnQueryDragIcon()
{
	return static_cast<HCURSOR>(m_hIcon);
}

// ================================================
//  INTEGRACIÓN MFC <-> OpenCV
// ================================================

// Convierte una cv::Mat en HBITMAP y la muestra dentro de IDC_PIC
void CMiEditorDlgDlg::ShowImage(const cv::Mat& src)
{
	if (src.empty()) return;

	CRect rc;
	m_pic.GetClientRect(&rc);
	if (rc.Width() <= 0 || rc.Height() <= 0) return;

	// Ajustar al tamaño del control manteniendo la proporción
	double scale = (std::min)((double)rc.Width()  / src.cols,
				  (double)rc.Height() / src.rows);
	cv::Mat resized, bgr;
	cv::resize(src, resized, cv::Size(), scale, scale, cv::INTER_AREA);

	// Siempre 3 canales (por si la imagen es gris o de bordes)
	if (resized.channels() == 1)
		cv::cvtColor(resized, bgr, cv::COLOR_GRAY2BGR);
	else
		bgr = resized;

	// Cada fila de un DIB debe alinearse a 4 bytes
	int w = bgr.cols, h = bgr.rows;
	int stride = ((w * 3 + 3) / 4) * 4;
	std::vector<uchar> buf((size_t)stride * h, 0);
	for (int y = 0; y < h; y++)
		memcpy(&buf[(size_t)y * stride], bgr.ptr(y), (size_t)w * 3);

	BITMAPINFO bmi = {};
	bmi.bmiHeader.biSize        = sizeof(BITMAPINFOHEADER);
	bmi.bmiHeader.biWidth       = w;
	bmi.bmiHeader.biHeight      = -h;   // negativo = de arriba hacia abajo
	bmi.bmiHeader.biPlanes      = 1;
	bmi.bmiHeader.biBitCount    = 24;
	bmi.bmiHeader.biCompression = BI_RGB;

	CClientDC dc(this);
	HBITMAP hbm = CreateDIBitmap(dc.GetSafeHdc(), &bmi.bmiHeader,
				     CBM_INIT, buf.data(), &bmi, DIB_RGB_COLORS);
```

```
	// Asignar al control y liberar el mapa de bits anterior
	HBITMAP old = (HBITMAP)m_pic.SendMessage(STM_SETIMAGE, IMAGE_BITMAP,
(LPARAM)hbm);
	if (old) DeleteObject(old);
	m_pic.Invalidate(TRUE);
}

// Recalcula todo desde la imagen original según el estado de la interfaz
void CMiEditorDlgDlg::ApplyPipeline()
{
	if (m_original.empty()) return;

	cv::Mat m = m_original.clone();

	// Brillo y contraste: salida = alpha * entrada + beta
	double alpha = m_sliderContrast.GetPos() / 10.0;   // 0.1 a 3.0
	int    beta  = m_sliderBright.GetPos();            // -100 a 100
	m.convertTo(m, -1, alpha, beta);

	// Desenfoque gaussiano (el kernel debe ser impar)
	int k = m_sliderBlur.GetPos();
	if (k > 0)
		cv::GaussianBlur(m, m, cv::Size(2 * k + 1, 2 * k + 1), 0);

	if (m_gray)
		cv::cvtColor(m, m, cv::COLOR_BGR2GRAY);

	if (m_edges)
	{
		if (m.channels() == 3) cv::cvtColor(m, m, cv::COLOR_BGR2GRAY);
		cv::Canny(m, m, 80, 160);
	}

	if (m_flip)
		cv::flip(m, m, 1);   // espejo horizontal

	m_current = m;
	ShowImage(m_current);
}

// ---------------- Eventos de la interfaz ----------------
void CMiEditorDlgDlg::OnBnClickedBtnOpen()
{
	CFileDialog dlg(TRUE, nullptr, nullptr, OFN_FILEMUSTEXIST,
		_T("Imagenes|*.png;*.jpg;*.jpeg;*.bmp||"));
	if (dlg.DoModal() != IDOK) return;

	CT2A path(dlg.GetPathName());
	m_original = cv::imread(std::string(path));
	if (m_original.empty())
	{
		AfxMessageBox(_T("No se pudo cargar la imagen."));
		return;
	}
	m_gray = m_edges = m_flip = false;
```

```
	ApplyPipeline();
}

void CMiEditorDlgDlg::OnBnClickedBtnSave()
{
	if (m_current.empty()) return;

	CFileDialog dlg(FALSE, _T("png"), _T("resultado"),
		OFN_OVERWRITEPROMPT, _T("PNG|*.png|JPG|*.jpg||"));
	if (dlg.DoModal() != IDOK) return;

	CT2A path(dlg.GetPathName());
	if (cv::imwrite(std::string(path), m_current))
		AfxMessageBox(_T("Imagen guardada."));
	else
		AfxMessageBox(_T("No se pudo guardar la imagen."));
}

void CMiEditorDlgDlg::OnBnClickedBtnGray()  { m_gray  = !m_gray;  ApplyPipeline(); }
void CMiEditorDlgDlg::OnBnClickedBtnCanny() { m_edges = !m_edges; ApplyPipeline(); }
void CMiEditorDlgDlg::OnBnClickedBtnFlip()  { m_flip  = !m_flip;  ApplyPipeline(); }

void CMiEditorDlgDlg::OnBnClickedBtnReset()
{
	m_sliderBright.SetPos(0);
	m_sliderContrast.SetPos(10);
	m_sliderBlur.SetPos(0);
	m_gray = m_edges = m_flip = false;
	ApplyPipeline();
}

void CMiEditorDlgDlg::OnHScroll(UINT nSBCode, UINT nPos, CScrollBar* pScrollBar)
{
	CDialogEx::OnHScroll(nSBCode, nPos, pScrollBar);
	ApplyPipeline();
}
```

### 6.3 Las tres funciones clave

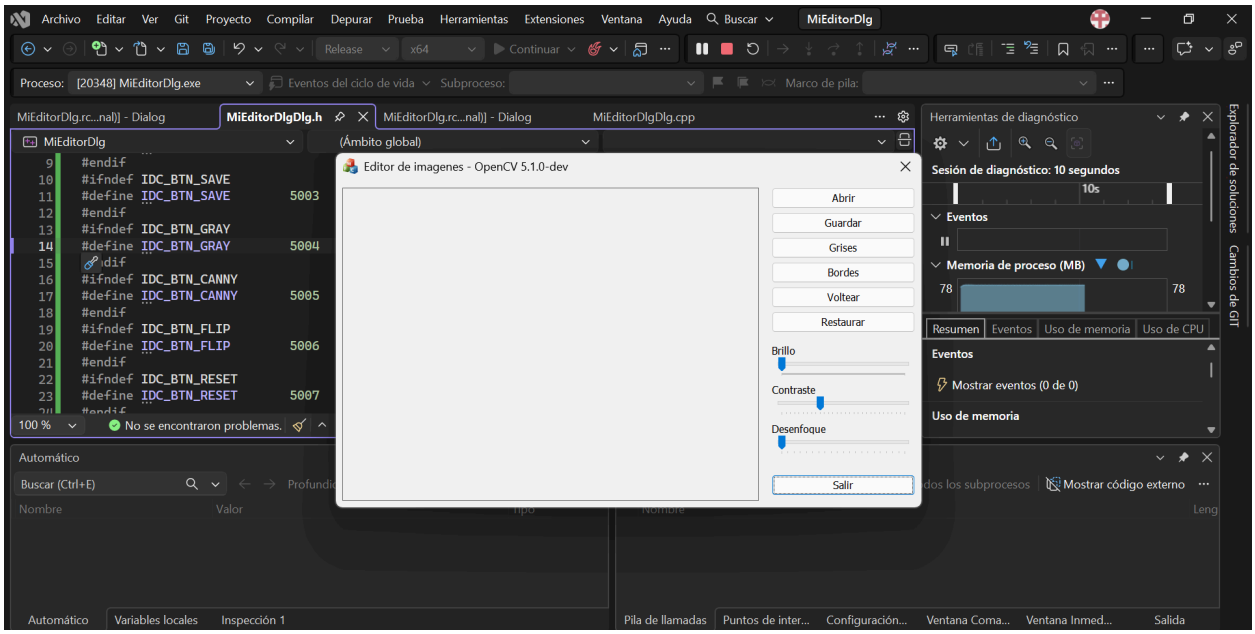
- **`ShowImage()`** convierte la `cv::Mat` en un `HBITMAP` y lo asigna al control `IDC_PIC` con `STM_SETIMAGE`. Redimensiona con `cv::resize` conservando la proporción y alinea cada fila a 4 bytes, como exige el formato DIB de Windows. No se usa `cv::imshow`, para que la imagen quede dentro de la interfaz.
- **`ApplyPipeline()`** parte siempre de `m_original` y aplica en orden brillo y contraste, desenfoque, grises, bordes y volteo, según el estado de los sliders y botones. Como recalcula desde el original, los efectos no se degradan al moverlos.
- **Manejadores de eventos** (`OnBnClicked...` y `OnHScroll`) solo cambian el estado (`m_gray`, `m_edges`, `m_flip`, posición de los sliders) y llaman a `ApplyPipeline()`.

## 7. Ejecución y pruebas

Con **Release | x64** seleccionado, se compila con **Ctrl+Shift+B** y se ejecuta con **Ctrl+F5**. Para las pruebas se usó la imagen `C:\img\encoder.png`.

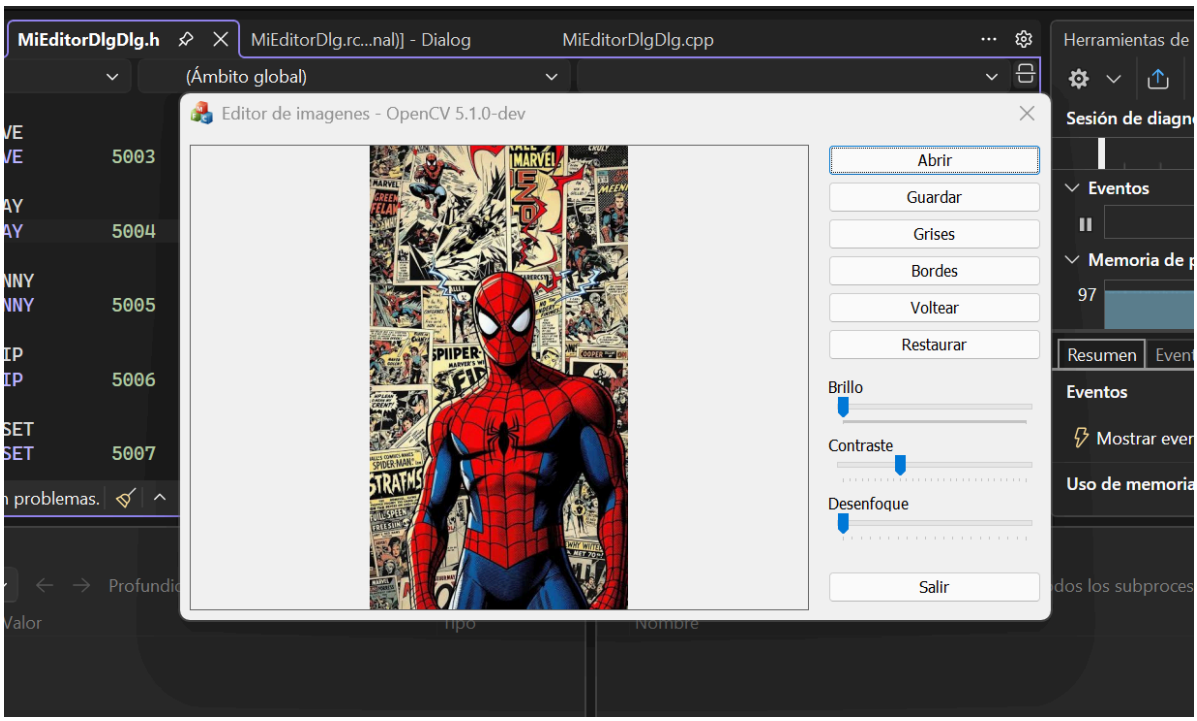
## 7.1 La aplicación en ejecución

Al abrir, la ventana muestra el cuadro de imagen a la izquierda, los controles a la derecha y, en la barra de título, la versión de OpenCV obtenida con `CV_VERSION`.



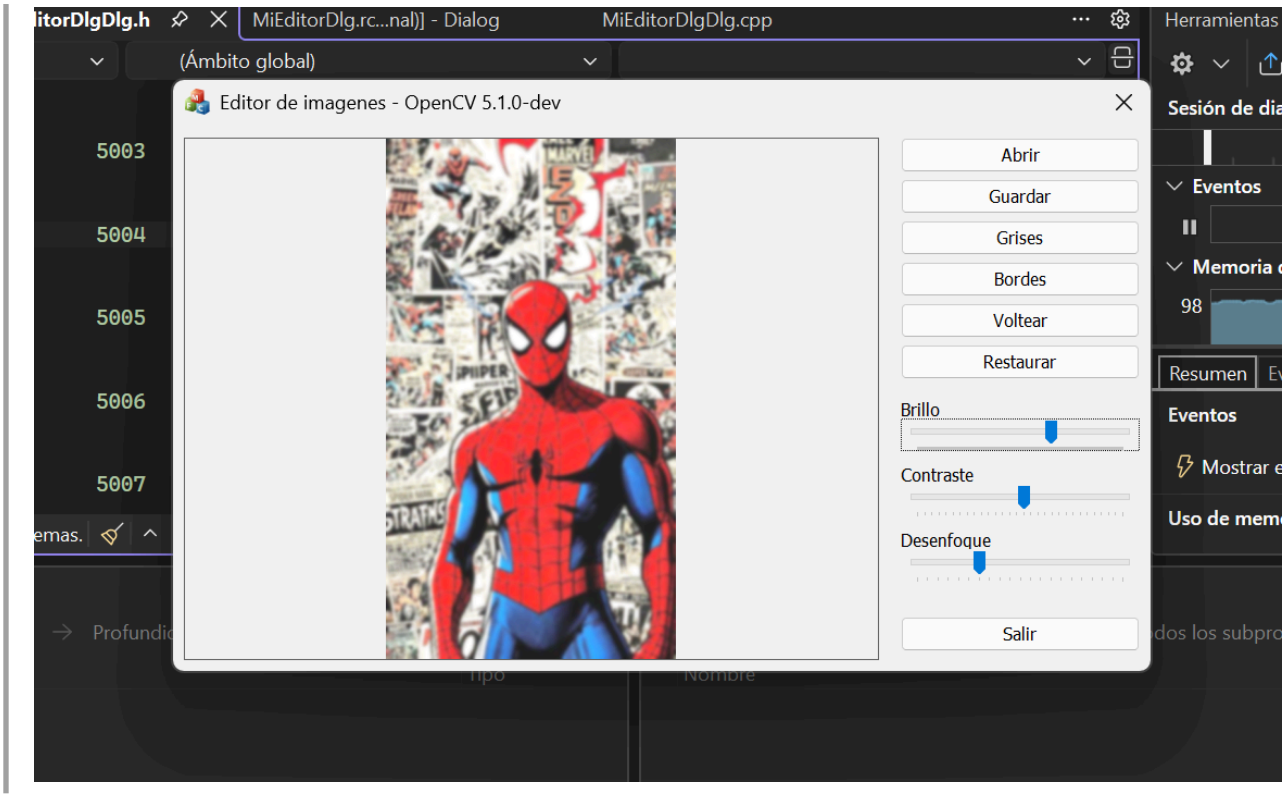

## 7.2 Abrir una imagen

El botón **Abrir** muestra el cuadro de diálogo de archivos (`CFileDialog`) y carga la imagen con `cv::imread`. Se muestra dentro del control mediante `ShowImage()`.



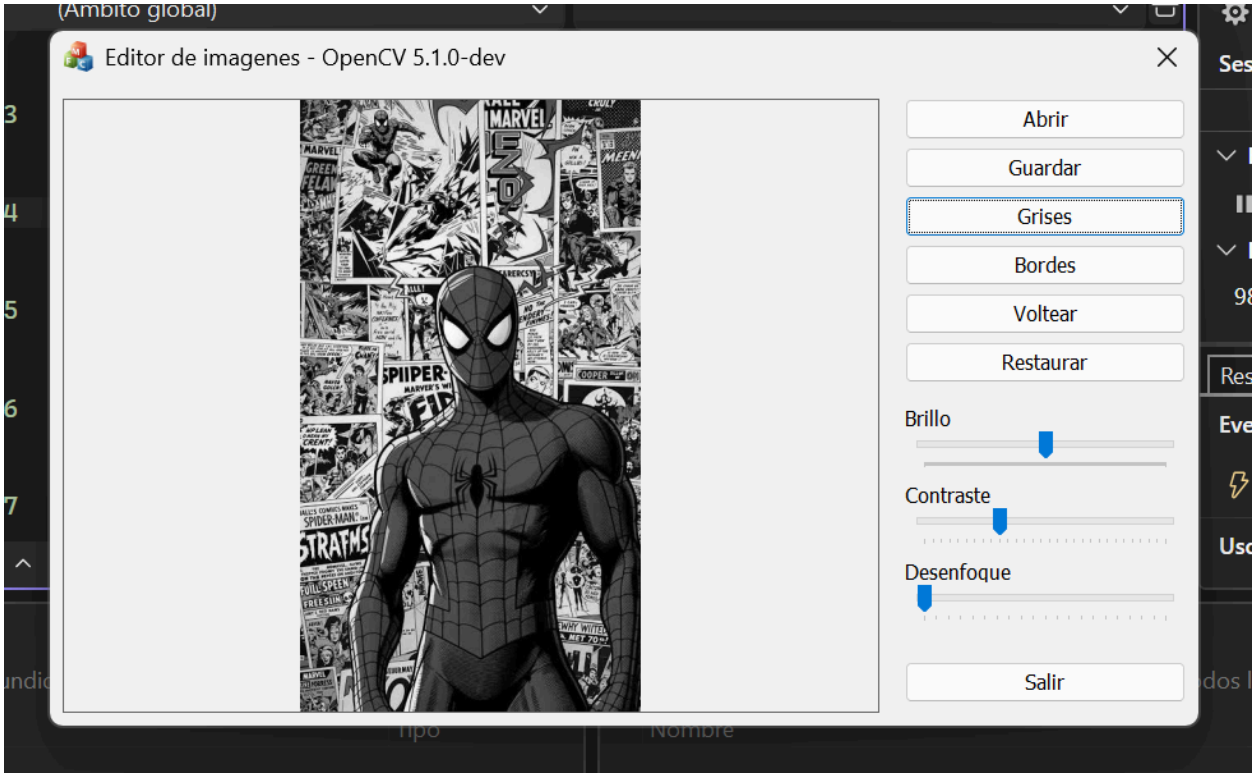
### 7.3 Brillo, contraste y desenfoque

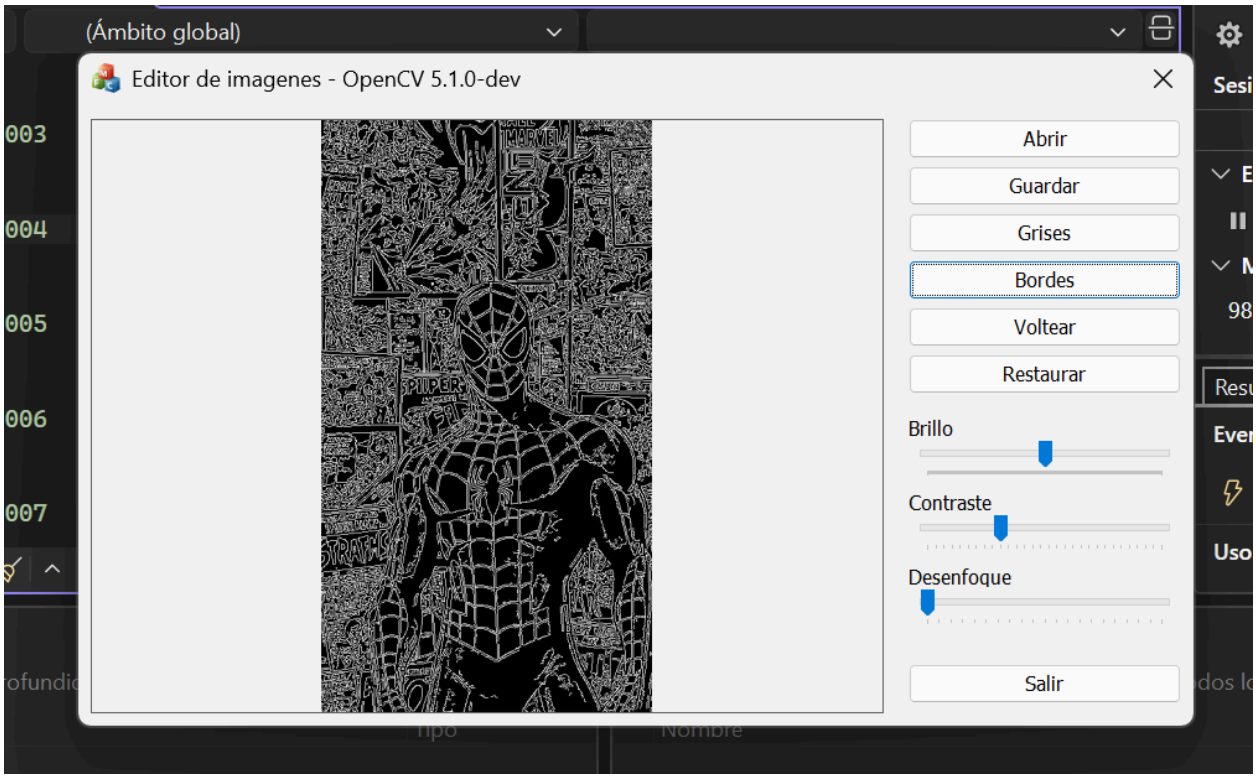
Los sliders envían `WM_HSCROLL`, y `ApplyPipeline()` recalcula la imagen con `convertTo` (brillo y contraste) y `cv::GaussianBlur` (desenfoque).



### 7.4 Filtros: grises y bordes

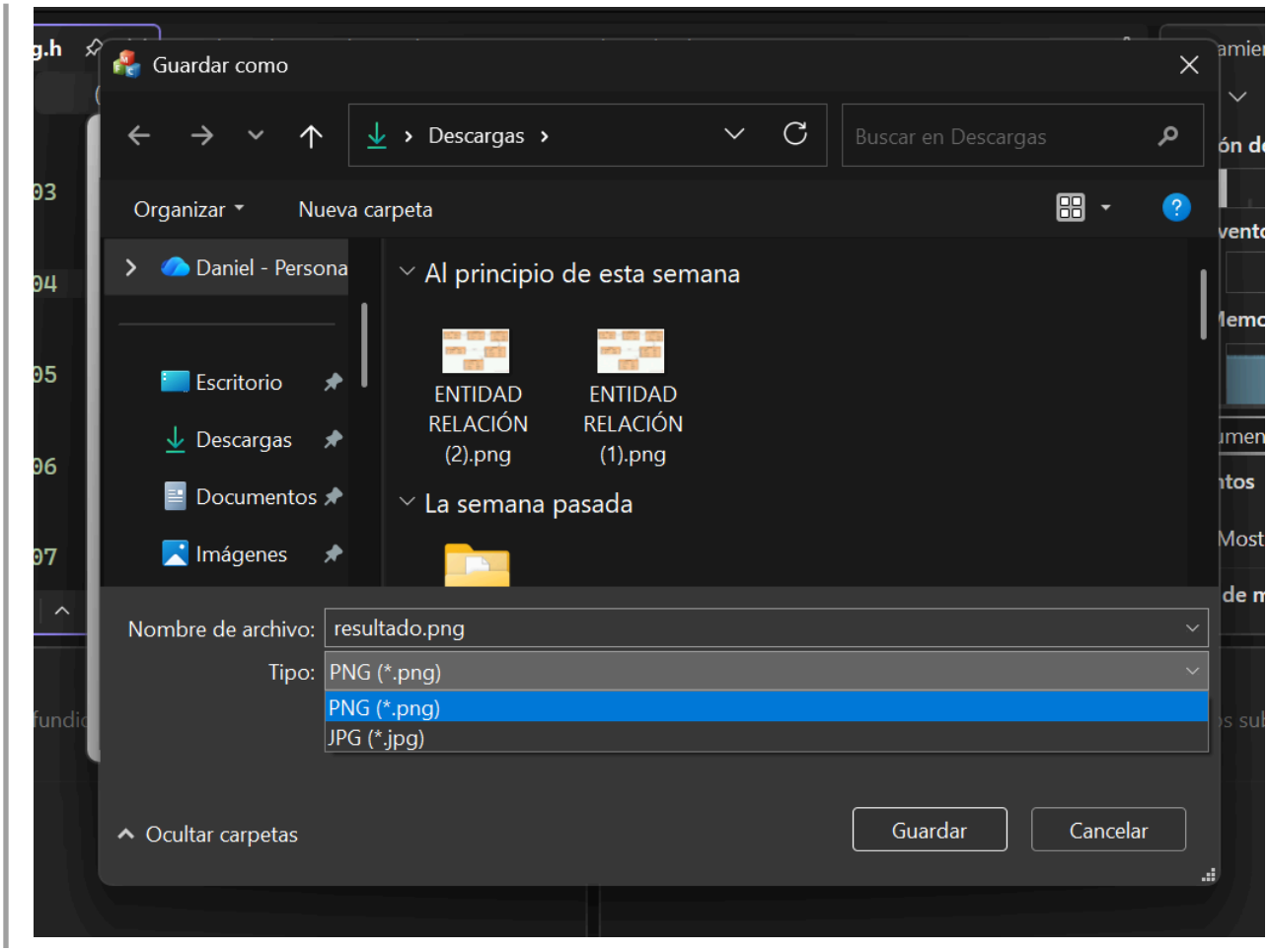
El botón **Grises** aplica `cv::cvtColor` y el botón **Bordes** aplica el detector de bordes `cv::Canny`.



### 7.5 Guardar el resultado

El botón **Guardar** escribe la imagen procesada con `cv::imwrite` en el formato elegido (PNG o JPG).




## 8. Interacción entre la interfaz y la librería

| Evento o elemento de la interfaz (MFC) | Función de OpenCV |
|---|---|
| Botón Abrir (`CFileDialog`) | `cv::imread` |
| Botón Guardar | `cv::imwrite` |
| Sliders de brillo y contraste (`WM_HSCROLL`) | `Mat::convertTo` |
| Slider de desenfoque | `cv::GaussianBlur` |
| Botón Grises | `cv::cvtColor` |
| Botón Bordes | `cv::Canny` |

| Evento o elemento de la interfaz (MFC) | Función de OpenCV |
|---|---|
| Botón Voltear | `cv::flip` |
| Cualquier cambio en pantalla | `cv::resize` y conversión de `cv::Mat` a `HBITMAP` para pintar en `IDC_PIC` |
| Barra de título | `CV_VERSION` |

Flujo general: el usuario modifica un control, MFC lanza el evento, `ApplyPipeline()` aplica las funciones de OpenCV sobre una copia de la imagen original y `ShowImage()` convierte el resultado en un mapa de bits que se muestra en el diálogo.

## 9. Problemas encontrados y soluciones

| Síntoma | Causa y solución |
|---|---|
| `cmake` no se reconoce en PowerShell | CMake no estaba instalado o no estaba en el PATH. Se instaló con `winget install Kitware.CMake` y se reabrió PowerShell |
| Error `NativeCommandError` al clonar | PowerShell 5.1 trata la salida de `git` como error. Se usa `$ErrorActionPreference = "Continue"` |
| El proyecto tenía `ChildFrm`, `MainFrm` y `MiEditorView` | Se había elegido una aplicación de múltiples documentos (MDI). Se recreó como **Basado en cuadro de diálogo** |
| `el identificador "m_pic" no está definido` y similares | `MiEditorDlgDlg.h` no se había reemplazado. Los miembros se declaran en el `.h` |
| `IDC_PIC` o `IDC_BTN_OPEN` no definidos | Los controles no existían en el editor de recursos. Se resolvió definiendo los IDs en el encabezado y creando los controles por código |
| `LNK1104` o `LNK2019` | Compilación en Debug o x86, o nombre de `.lib` incorrecto. Se usa Release \| x64 y `opencv_world510.lib` |
| Errores con `min` o `max` | Falta `NOMINMAX` y que OpenCV vaya antes de `framework.h` en `pch.h` |
| Al ejecutar: falta `opencv_world510.dll` | Se copia la DLL junto al `.exe` (sección 4.4) |
| Advertencias de `cvflann` o `LshStats` en la Lista de errores | Son avisos del análisis de código sobre los encabezados de OpenCV, no errores de compilación |

## 10. Conclusiones

En este proyecto compilé OpenCV 5.1 con opencv_contrib desde su código fuente, usando CMake y Visual Studio 2026, y la integré en una aplicación MFC. Compilar la librería en lugar de descargarla ya hecha me permitió entender cómo se relacionan el código fuente, la biblioteca (`.lib`), la DLL y las rutas de inclusión y enlace, y por qué el proyecto debe usar la misma configuración (Release | x64) con la que se compiló OpenCV.

Como mejoras futuras se podrían agregar más filtros, deshacer de varios niveles, zoom y recorte de la imagen.

## 11. Referencias


- OpenCV, documentación oficial: https://docs.opencv.org
- OpenCV, repositorio: https://github.com/opencv/opencv
- opencv_contrib, repositorio: https://github.com/opencv/opencv_contrib
- CMake, documentación: https://cmake.org/documentation
- Microsoft, documentación de MFC: https://learn.microsoft.com/cpp/mfc/mfc-desktop-applications